

Topic 4: Data Cleaning and Preparation

Splitting and merging

Igor Razgon

igor.razgon@durham.ac.uk

Content

1. Splitting data frames
2. Merging data frames

Splitting data frames

Splitting DataFrame into several parts

- In practice, it is often important to split the given dataset into several fragments (clusters) so that all the rows of each fragment have the same value of some key.
- For example, in organizational sociology, salary distributions are studied for a specific gender or for a specific age group.
- We are going to consider splitting of a DataFrame using *groupby* operation.

Connection to conditional probabilities

- In probability theory it is common to create a new probability space by imposing a condition.
- The rows of the given DataFrame can be seen as elements of a probability space.
- Suppose that one of the columns of the data frame is 'Gender'.
- Then considering only the fragment consisting of rows where 'Gender'='Female', we in fact impose the corresponding condition on the probability space.

groupby function

- *groupby* function splits a DataFrame instance into fragments.
- The function returns an object of type *GroupBy*.
- This object does not enable reaching the fragment through attributes.
- We need to use *getter* functions or *iterations* instead.

Key for *groupby*

- In order to run *groupby()* we identify a **key**.
- Two rows get into the same fragment if and only if they have the same value of the key.
- We will consider two ways for defining a key: values of a specific column or values of a function applied to a specific column

Accessing individual fragments

1. Iteration through all fragments.
2. Direct access through *get_group* function.

Example of iteration through all fragments

```
testdict={  
    'key1': ['a','b','a','a','a','b','a','b'],  
    'key2': ['c','c','c','c','d','d','d','d'],  
    'num1':[1,2,3,4,5,6,7,8],  
    'num2':[40,35,30,25,20,15,10,5]  
}
```

```
pdtest=pd.DataFrame(testdict)
```

```
pdgroups=pdtest.groupby(['key1'])
```

#Exploring all keys and the corresponding groups in a loop

for key,group in pdgroups:

print("\nKey =",key)

print(group)

Key = a

	key1	key2	num1	num2
0	a	c	1	40
2	a	c	3	30
3	a	c	4	25
4	a	d	5	20
6	a	d	7	10

Key = b

	key1	key2	num1	num2
1	b	c	2	35
5	b	d	6	15
7	b	d	8	5

Example of direct access through *get_group* function

```
testdict={  
    'key1': ['a','b','a','a','a','b','a','b'],  
    'key2': ['c','c','c','c','d','d','d','d'],  
    'num1':[1,2,3,4,5,6,7,8],  
    'num2':[40,35,30,25,20,15,10,5]  
}
```

```
pdtest=pd.DataFrame(testdict)
```

```
pdgroups=pdtest.groupby(['key1'])
```

#Getting the group corresponding to an individual key

```
subframe=pdgroups.get_group('a')
```

```
subframe
```

	key1	key2	num1	num2
0	a	c	1	40
2	a	c	3	30
3	a	c	4	25
4	a	d	5	20
6	a	d	7	10

groupby with accumulation

- We are going to consider application of *groupby()* in the form *groupby(..).f()* where *f* is an accumulating function like *sum*, *mean* or *max*.
- The result is the DataFrame indexed by the values of the key.
- For a key value *x*, the corresponding row is the result of function *f* applied to each column (but the key column) of the DataFrame corresponding to *x*.

Removal of duplicates for exams with resits

- Consider a DataFrame with two columns: student ID and exam mark.
- The same student ID may appear several times meaning that the student made several exam attempts.
- We want to leave only *one* row per student: the row with the *maximal* mark.
- Recall that the function *drop_duplicates()* did not provide such a possibility.

Removal of duplicates for exams, leaving maximal marks using *groupby()*

```
pdrepeat=pd.read_csv('examsrepeat.csv')
```

```
pdsummary=pdrepeat.groupby(['id']).max()
```

```
pdrepeat
```

```
#Aggregate quesry per column indexed by id
```

```
pdsummary
```

		id	exam
	0	15	40
	1	27	15
	2	27	90
	3	36	64
	4	111	56
	5	36	15
	6	111	65

One deficiency of the proposed solution with *groupby()*

- The input DataFrame has *two* columns 'id' and 'exam', while the output DataFrame has only *one* column 'exam'
- Column 'id' becomes the *index*.
- So, this is not exactly removal of duplicates since the DataFrame *format is changed*.
- A completely neat removal is achieved by using a function as a key.

Using function as a *groupby* key

- A standard format *df.groupby(mf)*, where the parameter *mf* is a function of a single argument.
- *mf* is applied to each element of *df.index*
- The data frame *df* is split into fragments, two rows get to the same fragment if and only if *mf* returns the same value for their respective indices.
- The application of the function is not completely straightforward as *mf* sees only the index, not the whole row.

Splitting student records according to the honours class of the Maths exam

- We again consider the student marks `DataFrame`.
- We want the function $mf(ind)$ to return the class of the mark column in row with index ind .
- Recall that we have function $MarkstoClasses$. However, the function needs the mark, not the row index.
- This can be done by specifying mf as
 $\lambda ind: MarkstoClasses(pdmarks['Maths'][ind]))$,
where $pdmarks$ is the *DataFrame* to which *groupby* is applied.

Implementation of student record splitting

```
pdmarks = pd.read_csv('student_marks.csv')
```

```
pdmarks
```

#Note the trick how function whose argument is index passes value to another function

#taking an argument from column 'Maths'

```
groupmarks=pdmarks.groupby(lambda ind: MarkstoClasses(pdmarks['Maths'][ind]))
```

```
for key,group in groupmarks:
```

```
    print("\nKEY =",key)
```

```
    print(group)
```

KEY = 1

	Name	Gender	DOB	Maths	Physics	Chemistry	English	Biology
1	Suresh	M	04/05/1987	75	96	78	64	90
3	Jessica	F	12/08/1990	78	96	86	63	54
9	Mukesh	M	02/09/1989	96	46	77	83	58

KEY = 2:2

	Name	Gender	DOB	Maths	Physics	Chemistry	English	Biology
0	John	M	05/04/1988	55	45	56	87	21
4	Jennifer	F	02/09/1989	58	96	78	46	96
6	pooja	F	04/05/1987	55	64	61	58	75
7	Ritesh	M	25/05/1989	54	76	87	56	25
8	Farha	F	12/08/1990	55	63	89	75	78

KEY = 3

	Name	Gender	DOB	Maths	Physics	Chemistry	English	Biology
5	Annu	F	05/04/1988	45	87	52	89	55

KEY = Fail

	Name	Gender	DOB	Maths	Physics	Chemistry	English	Biology
2	Ramesh	M	25/05/1989	25	54	89	76	95

Back to removal of duplicates for exams with resits

- Instead of using `['id']` as the key, we use the function:
lambda ind: pdrepeat['id'][ind],
where *pdrepeat* is the name of the *DataFrame* instance to which *groupby* is applied.
- As a result, we get the same format as the input data frame, but the index itself is distorted.
- We fix that by applying *reset_index(drop=True)* to the output of *groupby(...).max()* operation.

Example: removal of duplicates for exams with resits

```
pdrepeat=pd.read_csv('examsrepeat.csv')
pdrepeat
#Removal of duplicates leaving the largest remaining mark.
#The index becomes distorted as a result
pdsummary=pdrepeat.groupby(lambda ind: pdrepeat['id'][ind]).max()
pdsummary
pdsummary_stin=pdsummary.reset_index(drop=True)
pdsummary_stin
```

	id	exam
0	15	40
1	27	90
2	36	64
3	111	65

	id	exam
15	15	40
27	27	90
36	36	64
111	111	65

	id	exam
0	15	40
1	27	15
2	27	90
3	36	64
4	111	56
5	36	15
6	111	65

Testing variable dependencies

- Let us treat rows of a DataFrame *df* as elements of a uniform probability space.
- The columns can be seen as random variables
- For a column *col*, the value of the corresponding variable on row with index *x* is *df[col][x]*.
- We are going to use *groupby()* to design a function *DepTest(df,var1,var2)*, where
 - *df* is a *DataFrame* instance
 - *var1*, *var2* are column names of *df*.
- The function returns:
 - *True*, if *var1* and *var2* are *independent*
 - *False*, otherwise.

Practical importance of testing variable dependencies

- Studying dependencies within a data frame is important for studying standards of Equality, Diversity, and Inclusion at a specific organization.
- For example, we may study distribution of roles depending on the gender.

A high-level description

- We apply *df.groupby([var1])*
- We loop over the resulting fragments and verify that the column *var2* has the same distribution of values in all fragments.

Two aspects of the above strategy need to be further clarified.

- How to compute distribution of column values?
- How to test equality of distributions?

Missing aspects of the algorithm

- If we run `value_counts()` with parameter *normalize* set to *True*, the series will contain relative frequencies of values rather than the occurrence numbers. This is exactly the distribution of values that we need.
- The command `a.equals(b)`, where *a* and *b* are *Series* or *DataFrame* instances,
 - returns *True*, if *a* equals *b*,
 - returns *False*, otherwise
- Rather than comparing all the distributions with each other, we store aside the first one and compare the remaining distributions with the stored one. This way we make the number of comparisons linear in the number of *DataFrame* fragments.

Code of the function DepTest

#Function for dependency test of two columns

```
def DepTest(df,var1,var2):  
    groupvars=df.groupby([var1])  
    stored=False  
    for key,group in groupvars:  
        counts=group[var2].value_counts(normalize=True)  
        if not stored:  
            #Storing the first distribution  
            storedcounts=counts.copy()  
            stored=True  
        else:  
            #Comparing the distribution of the current fragment against the stored one  
            if not storedcounts.equals(counts):  
                return False  
    return True
```

Example: dependencies testing

```
pdvars = pd.read_csv('dependencies.csv')
```

```
pdvars
```

```
#Testing dependency between columns var1 and var2
```

```
var1var2dep=DepTest(pdvars,'var1','var2')
```

```
var1var2dep  the output is True
```

```
#Testing dependency between columns var1 and var3
```

```
var1var3dep=DepTest(pdvars,'var1','var3')
```

```
Var1var3dep  the output is False
```

	var1	var2	var3
0	1	1	1
1	1	1	1
2	1	2	1
3	1	2	2
4	1	3	2
5	2	1	1
6	2	1	1
7	2	1	1
8	2	1	2
9	2	2	2
10	2	2	2
11	2	2	2
12	2	2	2
13	2	3	2
14	2	3	2

Merging data frames

Merging DataFrames

- Merging two data frames is like taking a Cartesian product of two sets.
- The rows of the new DataFrame are all possible concatenations of a row of the *first* DataFrame and a row of the *second* DataFrame.
- The merged data frames must have at least one column in common.
- Two rows concatenate if and only if their respective values in the overlapping columns match.

Why do we need merging DataFrames?

- The reason is exactly the same as the for the use of *join* operation for databases.
- Due to various reasons, DataFrame is stored not explicitly but decomposed into several parts.
- One such reason may be that different parts are produced by different organizations.
- The parts then need to be merged for the purpose of data analysis.

Examples of merging of data frames

```
dict1={  
  'numbers':[1,2,3,4],  
  'connect':[1,1,2,7]  
}
```

```
dict2={  
  'connect':[2,2,1,8],  
  'letters':['a','b','c','d']  
}
```

```
df1=pd.DataFrame(dict1)
```

```
df2=pd.DataFrame(dict2)
```

#Inner merge

```
df=pd.merge(df1,df2)
```

df

#Outer merge

```
dfouter=pd.merge(df1,df2,how='outer')
```

dfouter

	numbers	connect	letters
0	1	1	c
1	2	1	c
2	3	2	a
3	3	2	b

	numbers	connect	letters
0	1.0	1	c
1	2.0	1	c
2	3.0	2	a
3	3.0	2	b
4	4.0	7	NaN
5	NaN	8	d

A word of caution: size blow up

- The size of the merge of two data frames maybe close to the product of sizes of DataFrames being merged.
- For example, the result of merging of two data frames of a million rows each (a *reasonable* size) may be a DataFrame of about a **trillion** rows, which is *completely unreasonable*.
- Therefore, a data analyst, needs to weight the need to merge DataFrames and, if possible to carry out the analysis of the *decomposed* representation.